FRESHCART ENTERPRISE ARCHITECTURE

Jakarta EE 10 Revision & Architecture Note

A practical, code-grounded reference manual based on the **FreshCart Grocery Project**, covering Spring Boot 3, Microsoft SQL Server, Redis In-Memory Caching, CDI 4.0 Scopes, JPA 3.1, Bean Validation, and JTA Transactions.

Author / Candidate: Virendra Arekar	**Stack:** Spring Boot 3.3.4 (Jakarta EE 10)
Persistence: MS SQL Server 2022 (ACID)	**In-Memory Caching:** Redis 7.x (Lettuce/JSON)
Deployment: Docker Compose & Kubernetes	**Frontend:** Next.js 14 App Router + Tailwind

1. The Jakarta EE Evolution & Namespace Migration

In 2017, Oracle transferred the stewardship of **Java EE** to the **Eclipse Foundation**. Because the 'Java' trademark remained with Oracle, the enterprise standard was rebranded as **Jakarta EE**. Starting with Jakarta EE 9 and enforced across Spring Boot 3.0+, all traditional javax.* package declarations were permanently migrated to the jakarta.* namespace.

Specification Area	Legacy Java EE (<= 8)	Modern Jakarta EE 10/11	FreshCart Implementation
Persistence (JPA)	`javax.persistence.*`	`jakarta.persistence.*`	BaseEntity, Product, Category, Order
Bean Validation	`javax.validation.*`	`jakarta.validation.*`	@NotNull, @NotBlank, @DecimalMin
CDI & Scopes	`javax.enterprise.context.*`	`jakarta.enterprise.context.*`	@ApplicationScoped, @RequestScoped
Transactions (JTA)	`javax.transaction.*`	`jakarta.transaction.*`	@Transactional(rollbackOn=Exception)
Servlet Spec	`javax.servlet.*`	`jakarta.servlet.*`	Tomcat 10.1 Web Server Engine

CRITICAL ARCHITECTURAL RULE:

Spring Boot 3.x strictly rejects legacy javax.persistence imports. Attempting to use javax. annotations in Spring Boot 3 will cause silent ignore or ClassNotFoundException. All FreshCart entities use jakarta.* exclusively.*

2. Contexts and Dependency Injection (CDI) & Bean Scopes

CDI manages bean lifecycles, contextual proxies, and loose coupling. In FreshCart, we implemented an interactive live diagnostic endpoint (/api/jakarta-guide/scopes/live) that demonstrates how different bean scopes behave in real time.

Jakarta CDI Scope	Spring Equivalent	Lifecycle & Boundary	Concurrency & Thread Safety
`@ApplicationScoped`	@ApplicationScope / Singleton	Matches JVM lifetime; shared across all users/threads	**Must be Thread-Safe** (Concurrent reads/writes)
`@RequestScoped`	@RequestScope	Bound to a single HTTP request-response cycle	Thread-safe by default (confined to request thread)
`@SessionScoped`	@SessionScope	Bound to HTTP session across multiple requests	**Must implement java.io.Serializable**
`@ConversationScoped`	Custom scope / Flash	Explicit boundaries via Conversation.begin() / end()	**Must implement java.io.Serializable**
`@Dependent (Default)`	@Scope('prototype')	Tied directly to the lifecycle of the injecting bean	No proxy; shares caller's concurrency model

CDI Client Proxies vs Pseudo Scopes:

Normal scopes (@ApplicationScoped, @RequestScoped, @SessionScoped) inject a bytecode client proxy instead of a direct reference. This allows injecting a short-lived bean (like a request audit token) into a long-lived application singleton safely; the proxy intercepts invocations and delegates to the thread-local instance of the active request.

3. Jakarta Persistence API (JPA 3.1) in FreshCart

JPA provides object-relational mapping onto Microsoft SQL Server. The FreshCart entity model demonstrates production patterns for auditing, locking, inheritance, and cascade deletion.

3.1 BaseEntity — Auditing & Optimistic Locking

BaseEntity utilizes @MappedSuperclass so child entities inherit primary key, versioning, and timestamp hooks without an unnecessary parent table.

```
@MappedSuperclass
public abstract class BaseEntity implements Serializable {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Version
    @Column(name = "version")
    private Long version;  // Optimistic locking field

    @Column(name = "created_at", nullable = false, updatable = false)
    private LocalDateTime createdAt;

    @PrePersist
    protected void onCreate() { this.createdAt = LocalDateTime.now(); }

    @PreUpdate
    protected void onUpdate() { this.updatedAt = LocalDateTime.now(); }
}
```

3.2 Entity Associations & Cascade Semantics

FreshCart models parent-child relationships between **Category** ↔ **Product** and **Order** ↔ **OrderItem**:

- **Owning Side:** Product contains @ManyToOne(fetch = FetchType.EAGER) and holds the physical foreign key column via @JoinColumn(name = 'category_id').
- **Inverse Side:** Category defines @OneToMany(mappedBy = 'category', cascade = CascadeType.ALL, orphanRemoval = true).
- **orphanRemoval = true vs CascadeType.REMOVE:** CascadeType.REMOVE only deletes children when the parent itself is deleted. orphanRemoval = true detects when a child is removed from the Java collection (e.g., order.getItems().remove(item)) and automatically issues a SQL DELETE statement for that detached child.
- **@Enumerated(EnumType.STRING):** Used in Product.unit and Order.status. *Always avoid EnumType.ORDINAL*; reordering or inserting enum constants causes silent data corruption in the database.

3.3 Concurrency & Locking Strategies: Optimistic vs Pessimistic

Locking Type	Annotation / Mechanism	Database SQL Behavior	FreshCart Use Case
Optimistic Locking	`@Version private Long version;`	UPDATE ... WHERE id = ? AND version = ? Throws OptimisticLockException if 0 rows modified	Catalog browsing and general product detail edits (low conflict rate)
Pessimistic Locking	`@Lock(LockModeType.PESSIMISTIC_WRITE)`	SELECT ... WITH (UPDLOCK, ROWLOCK) Direct exclusive row lock in MS SQL Server	Checkout order stock decrement (eliminates negative inventory overselling)

4. Jakarta Bean Validation 3.0

Declarative constraint validation is executed at REST controller entry (via @Valid) and verified before EntityManager commit.

Constraint	Target Field in FreshCart	Validation Behavior
`@NotNull`	Product.price, Product.stockQuantity	Field must not be null (permits empty string)
`@NotBlank`	Product.name, Customer.fullName	Must not be null and must contain >= 1 non-whitespace character
`@Size(min, max)`	Category.name (2-50 chars)	Enforces minimum and maximum length boundaries
`@DecimalMin('0.01')`	Product.price, Order.totalAmount	Monetary boundary check for BigDecimal instances
`@PositiveOrZero`	Product.stockQuantity	Prevents negative inventory levels in product stock
`@Email`	Customer.email	Verifies valid RFC 5322 email syntax
`@Valid`	OrderRequestDTO.items (List)	**Cascades validation** into each nested OrderItemRequest element

5. Jakarta Transactions (JTA) & Redis Cache Coherence

Order placement in OrderServiceImpl.java combines atomic transaction boundaries with real-time Redis cache eviction:

```
@Service
public class OrderServiceImpl implements OrderService {
    @Override
    @Transactional(rollbackOn = Exception.class)
    @CacheEvict(value = {"productList", "products"}, allEntries = true)
    public OrderResponseDTO placeOrder(OrderRequestDTO request) {
        // 1. Fetch or create Customer
        // 2. Lock each Product with PESSIMISTIC_WRITE & decrement stock
        // 3. Persist Order and cascade OrderItems
        // 4. Redis cache is evicted so storefront reflects updated stock
    }
}
```

- **Atomicity & Rollback:** If any product has insufficient stock, an InsufficientStockException is thrown. @Transactional(rollbackOn = Exception.class) rolls back all database operations, restoring customer and stock state.
- **Cache Invalidation:** @CacheEvict(value = {'productList', 'products'}, allEntries = true) automatically invalidates stale in-memory cached responses in Redis 7, ensuring clients receive immediate consistent inventory counts.

6. Docker & Kubernetes Data Durability Architecture

Both Microsoft SQL Server and Redis are configured to ensure zero data loss across container teardowns and Kubernetes pod rescheduling:

- **Docker Compose Persistence:** Named volume mssql_data:/var/opt/mssql safeguards SQL Server tables, clustered indexes, and catalog records across docker compose down. Redis utilizes redis_data:/data with Append-Only File (AOF) logging (--appendonly yes).
- **Kubernetes Persistent Volumes:** mssql-pvc.yaml allocates 10Gi of durable storage with ReadWriteOnce access. mssql-deployment.yaml specifies strategy: Recreate to guarantee safe unmount/remount operations.

7. Top 10 Jakarta EE Technical Interview Questions

Q1: What causes the N+1 SELECT problem in JPA and how do you resolve it?

The N+1 problem occurs when fetching an entity with lazy collections. Loading 100 Categories issues 1 initial SQL query, but accessing products on each issues 100 additional queries (1 + N = 101 queries). Resolution: Use JPQL JOIN FETCH (e.g. SELECT c FROM Category c LEFT JOIN FETCH c.products) or declare an @EntityGraph.

Q2: Why did Java EE change to Jakarta EE and what are the package implications?

Oracle transferred Java EE to the Eclipse Foundation in 2017 but retained the legal trademark on the name 'Java'. Consequently, the specification was renamed Jakarta EE. In Jakarta EE 9, all package namespaces transitioned from javax.* to jakarta.*. Spring Boot 3 requires Jakarta EE 10.

Q3: Explain the difference between @ApplicationScoped and @Singleton in CDI.

@ApplicationScoped is a normal scope that injects a client proxy, supporting decorators, interceptors, and circular dependencies. @Singleton is a pseudo-scope that injects a direct bean reference without a proxy wrapper.

Q4: How does Optimistic Locking with @Version prevent lost updates?

Hibernate compares the entity version column during UPDATE (WHERE id = ? AND version = ?). If another transaction committed an update first, the version mismatch results in 0 rows updated, and JPA throws OptimisticLockException without holding database locks.

Q5: When would you choose Pessimistic Locking over Optimistic Locking?

When transaction contention is high and collision rollbacks would be expensive or unacceptable (e.g., flash-sale inventory deduction or banking balance updates). FreshCart uses LockModeType.PESSIMISTIC_WRITE to acquire exclusive UPDLOCK, ROWLOCK in MS SQL Server.

Q6: What is the difference between orphanRemoval = true and CascadeType.REMOVE?

CascadeType.REMOVE only deletes children when the parent entity itself is deleted. orphanRemoval = true detects when an entity is removed from a parent collection (order.getItems().remove(item)) and immediately issues a SQL DELETE for that child record.

Q7: Why must you avoid @Enumerated(EnumType.ORDINAL)?

ORDINAL stores the integer index of the enum constant. If enum constants are reordered, or a new constant is inserted in the middle, all existing database rows will silently point to the wrong enum value. EnumType.STRING stores the textual name and is safe against refactoring.

Q8: How does @Valid differ from @NotNull?

@NotNull validates that a reference is non-null. @Valid instructs the validation engine to cascade validation recursively into child objects or collection elements (e.g. validating each OrderItemRequest inside OrderRequestDTO).

Q9: What is the purpose of @MappedSuperclass?

Designates a base class whose mapping information (fields, IDs, timestamps, @PrePersist callbacks) is inherited by child entities. It does not generate an independent database table for the superclass itself.

Q10: What is the difference between jakarta.transaction.Transactional and Spring @Transactional?

jakarta.transaction.Transactional is the official JTA specification annotation. Spring's @Transactional is a proprietary Spring annotation offering additional capabilities such as readOnly = true, timeout, and custom transaction manager qualifiers. Both are supported in Spring Boot 3.